

MATH00041: Introduction to Programming

Tuples – Lists – Arrays

Lecture 3

Archontis Giannakidis, PhD

Outline

- **Tuples**
- **Lists**
- **Arrays**

Tuples

Tuples are a way of storing multiple items in a single variable in Python. A tuple is one of the four built-in data types in Python that are used to store collections of data. The other three data types are List, Set, and Dictionary, each with different properties and uses. Tuples are collections that are ordered and unchangeable, meaning that once created, their contents cannot be modified. Tuples are expressed using round brackets.

Example:

Create a Tuple:

```
new_tuple = ("apple", "banana", "cherry")  
print(new_tuple)
```

Characteristics of tuples and their explanation:

1. **Ordered:** This means that the items in a tuple have a defined order, and that order will not change. The first item in a tuple has an index of 0, and the second item has an index of 1, and so on. This is useful when you need to access specific items in a tuple.
2. **Unchangeable:** Tuples are unchangeable, meaning that you cannot change, add, or remove items after the tuple has been created. This makes tuples "immutable," whereas lists are "mutable." This characteristic of tuples makes them useful for situations where you need to store data that should not be changed accidentally.
3. **Allow Duplicates:** Since tuples are indexed, they can have items with the same value. This means that tuples can contain duplicate values, just like lists. However, unlike sets, tuples preserve the order of their items.

In summary, tuples are a data structure in Python that are ordered, unchangeable, and allow duplicates. You can use tuples effectively in your Python programs by understanding these characteristics.

Example:

Tuples allow duplicate values:

```
thistuple = ("apple", "banana", "cherry", "apple", "cherry")  
print(thistuple)
```

Output: ('apple', 'banana', 'cherry', 'apple', 'cherry')

Tuple Length

To determine how many items a tuple has, use the `len()` function:

Example:

Print the number of items in the tuple:

```
thistuple = ("apple", "banana", "cherry")  
print(len(thistuple))
```

Output:3

Create a Tuple with One Item

To create a tuple with only one item, you must add a comma after the item; otherwise, Python will not recognise it as a tuple.

Example:

One item tuple, remember the comma:

```
thistuple = ("apple",)  
print(type(thistuple))
```

#NOT a tuple

```
thistuple = ("apple")  
print(type(thistuple))
```

Output: <class 'tuple'>

Output: <class 'str'>

Tuple Items - Data Types

Tuple items can be of any data type:

Example:

Print the items in the tuple:

```
tuple1 = ("apple", "banana", "cherry")
```

```
tuple2 = (1, 5, 7, 9, 3)
```

```
tuple3 = (True, False, False)
```

```
Output: ('apple', 'banana', 'cherry')
```

```
Output: (1, 5, 7, 9, 3)
```

```
Output: (True, False, False)
```

A tuple can contain different data types:

Example:

A tuple with strings, integers and boolean values:

```
tuple1 = ("abc", 34, True, 40, "male")
```

```
Output: ('abc', 34, True, 40, 'male')
```

Tuples in Python are objects that are specifically defined as having the data type 'tuple':

```
<class 'tuple'>
```

Example:

Find the data type of a tuple:

```
mytuple = ("apple", "banana", "cherry")  
print(type(mytuple))
```

```
Output: <class 'tuple'>
```

The tuple() Constructor

It is also possible to use the tuple() constructor to make a tuple.

Example:

Using the tuple() method to make a tuple:

```
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets  
print(thistuple)
```

Output: ('apple', 'banana', 'cherry')

Access Tuple Items

You can access tuple items by referring to the index number, inside square brackets:

Example:

Print the second item in the tuple:

```
thistuple = ("apple", "banana", "cherry")  
print(thistuple[1])
```

Output: banana

❖ Note: The first item has index 0.

Negative Indexing

In Python, negative indexing is used to access elements in a sequence such as a string, list or tuple from the end. The index value -1 refers to the last item, -2 refers to the second last item, and so on. For example, if you have a list containing [1, 2, 3, 4, 5], then list[-1] will return 5, list[-2] will return 4, and so on.

Example:

Print the last item of the tuple:

```
thistuple = ("apple", "banana", "cherry")  
print(thistuple[-1])
```

Output: cherry

Range of Indexes

In Python, you can create a new tuple by specifying a range of indexes using the syntax ``tuple[start:end]``. This will return a new tuple with the items from the original tuple starting from the index ``start`` up to, but not including, the index ``end``.³

Example:

Return the third, fourth, and fifth item:

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[2:5])
```

Output: ('cherry', 'orange', 'kiwi')

- ❖ Note: The search will start at index 2 (included) and end at index 5 (not included).
- ❖ Note: Remember that the first item has index 0.

By leaving out the start value, the range will start at the first item:

Example:

This example returns the items from the beginning to, but NOT included, "kiwi":

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[:4])
```

Output: ('apple', 'banana', 'cherry', 'orange')

By leaving out the end value, the range will go on to the end of the list

Example:

This example returns the items from "cherry" and to the end:

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[2:])
```

Output: ('cherry', 'orange', 'kiwi', 'melon', 'mango')

Similar as the positive indexes in python it is possible to specify negative indexes if you want to start the search from the end of the tuple:

Example:

This example returns the items from index -4 (included) to index -1 (excluded)

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[-4:-1])
```

Output: ('orange', 'kiwi', 'melon')

- ❖ Note: Negative indexing means starting from the end of the tuple.
- ❖ Note: Remember that the last item has the index -1,

Check if Item Exists

To determine if a specified item is present in a tuple use the `in` keyword:

Example:

Check if "apple" is present in the tuple:

```
thistuple = ("apple", "banana", "cherry")  
if "apple" in thistuple:  
    print("Yes, 'apple' is in the fruits tuple")
```

Output: Yes, 'apple' is in the fruits tuple

Update Tuples

Tuples are immutable, meaning that once created, their contents cannot be changed. However, it's still possible to perform operations on tuples, such as concatenation or slicing.

Change Tuple Values

After creating a tuple, its values cannot be changed as tuples are immutable. However, you can convert a tuple into a list, modify the list, and then convert it back into a tuple. This way, you can indirectly change a tuple's values.

Example:

Convert the tuple into a list to be able to change it:

```
x = ("apple", "banana", "cherry")
```

```
y = list(x)
```

```
y[1] = "kiwi"
```

```
x = tuple(y)
```

Output: ("apple", "kiwi", "cherry")

Add Items

Since tuples are immutable, they do not have a built-in `append()` method, but there are other ways to add items to a tuple.

1. **Convert into a list:** Just like the workaround for changing a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

Example:

Convert the tuple into a list, add "orange", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")  
y = list(thistuple)  
y.append("orange")  
thistuple = tuple(y)
```

Output: ('apple', 'banana', 'cherry', 'orange')

2. Add tuple to a tuple: You can add tuples to each other. To do this, create a new tuple with the item(s) you want to add and append it to the existing tuple.

Example:

Create a new tuple with the value "orange", and add that tuple:

```
thistuple = ("apple", "banana", "cherry")  
y = ("orange",)  
thistuple += y  
print(thistuple)
```

Output: ('apple', 'banana', 'cherry', 'orange')

Remove Items

Tuples are unchangeable, so you cannot remove items from it, but you can use the same workaround as we used for changing and adding tuple items:

Example:

Convert the tuple into a list, remove "apple", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")  
y = list(thistuple)  
y.remove("apple")  
thistuple = tuple(y)
```

Output: ('banana', 'cherry')

Or you can delete the tuple completely:

Example:

The del keyword can delete the tuple completely:

```
thistuple = ("apple", "banana", "cherry")  
del thistuple  
print(thistuple) #this will raise an error because the tuple no longer exists
```

Unpacking a Tuple

When we create a tuple, we typically assign values to it. This is known as "packing" a tuple.

Example:

Packing a tuple:

```
fruits = ("apple", "banana", "cherry")
```

Output: ('apple', 'banana', 'cherry')

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking":

Example:

Unpacking a tuple:

```
fruits = ("apple", "banana", "cherry")
```

```
(green, yellow, red) = fruits
```

```
print(green)
```

```
print(yellow)
```

```
print(red)
```

Output: apple

Output: banana

Output: cherry

Using Asterisk*

If you have a list of values that you want to assign to a set of variables, but the number of variables is less than the number of values, you can use the `*` operator to assign the remaining values to a list variable.

Example:

Assign the rest of the values as a list called "red":

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
```

```
(green, yellow, *red) = fruits
```

```
print(green)
```

```
print(yellow)
```

```
print(red)
```

Output: apple

Output: banana

Output: ['cherry', 'strawberry', 'raspberry']

If an asterisk is added to a variable name other than the last one, Python will assign values to the variable until there are as many values left as there are variables left.

Example:

Add a list of values the "tropic" variable:

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")
```

```
(green, *tropic, red) = fruits
```

```
print(green)
```

```
print(tropic)
```

```
print(red)
```

Output: apple

Output: ['mango', 'papaya', 'pineapple']

Output: cherry

Join Two Tuples

To join two or more tuples, you can use the `+` operator:

Example:

Join two tuples:

```
tuple1 = ("a", "b" , "c")
```

```
tuple2 = (1, 2, 3)
```

```
tuple3 = tuple1 + tuple2
```

```
print(tuple3)
```

Output: ('a', 'b', 'c', 1, 2, 3)

Multiply Tuples

If you want to multiply the content of a tuple a given number of times, you can use the `*` operator:

Example:

Multiply the fruits tuple by 2:

```
fruits = ("apple", "banana", "cherry")
```

```
mytuple = fruits * 2
```

```
print(mytuple)
```

Output: ('apple', 'banana', 'cherry', 'apple', 'banana', 'cherry')

Tuple Methods

Python has two built-in methods that you can use on tuples.

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

Example:

Return the number of times the value 5 appears in the tuple:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
```

```
x = thistuple.count(5)  
print(x)
```

Output: 2

Example:

Search for the first occurrence of the value 8, and return its position:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
```

```
x = thistuple.index(8)  
print(x)
```

Output: 3

List

Lists are essential to store multiple items in a single variable in Python. There are four built-in data types that can be used to store collections of data in Python, which are Tuple, Set, Dictionary, and List. Each of these data types has unique characteristics and specific use cases. You can create a list in Python by using square brackets.

Example:

Create a List:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Output: ['apple', 'banana', 'cherry']

List Items

In Python, lists are a type of data structure that can store multiple values of different data types. They are ordered, which means that the order of the elements in the list is preserved. Lists are also mutable, so you can change the values of the elements in the list.

Lists allow duplicate values, which means that you can have multiple elements in the list with the same value. Each element in the list is assigned an index number, starting from 0 for the first element, 1 for the second element, and so on. You can access elements in the list using their index number.

Ordered

Lists being ordered means that the items in the list have a specific order that will not change. Whenever new items are added to the list, they get placed at the end of the list.

- ❖ Note: Some list methods will change the order, but generally, the order of the items will not change.

Changeable

In Python, when we create a list, it is by default a mutable data type. This means that we can modify the elements of the list after it has been created. For example, we can add or remove elements from the list, or change the value of existing elements.

Allow Duplicates

In Python, a list is a collection of items that can be of the same or different data types. Each item in a list is assigned a unique index number, starting from 0. This means that you can have multiple items in a list that have the same value, as long as they are assigned different index numbers.

Example:

Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]  
print(thislist)
```

Output: ['apple', 'banana', 'cherry', 'apple', 'cherry']

List Length

In Python, the `len()` function can be used to determine the number of items in a list. Simply pass the list as an argument to the `len()` function and it will return the number of items in the list.

Example:

Print the number of items in the list:

```
thislist = ["apple", "banana", "cherry"]  
print(len(thislist))
```

Output: 3

List Items - Data Types

In programming, a list is a collection of items or values that can be of any data type. For instance, a list can contain integers, floating-point numbers, strings, boolean values, or objects. Lists allow you to store multiple items of different types in a single variable.

Example:

String, int and boolean data types:

```
list1 = ["apple", "banana", "cherry"]
```

```
list2 = [1, 5, 7, 9, 3]
```

```
list3 = [True, False, False]
```

A list can contain different data types:

Example:

A list with strings, integers and boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
```

List type()

From Python's perspective, lists are defined as objects with the data type 'list': <class 'list'>

Example:

Print the number of items in the list:

```
mylist = ["apple", "banana", "cherry"]  
print(type(mylist))
```

Output: <class 'list'>

The list() Constructor

It is also possible to use the `list()` constructor when creating a new list.

Example:

Using the `list()` constructor to make a List:

```
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets  
print(thislist)
```

Output: ['apple', 'banana', 'cherry']

Access Items

List items are indexed, and you can access them by referring to the index number:

Example:

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1]) # The first item has index 0.
```

Output: banana

Negative Indexing

Negative indexing starts from the end. For instance, -1 refers to the last item, -2 refers to the second last item, and so on.

Example:

Print the last item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[-1])
```

Output: cherry

Range of Indexes

You can specify a range of indexes in a list by indicating the start and end points. When you specify a range, a new list will be returned containing the specified items.

Example:

Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[2:5]) # The search will start at index 2 (included) and end at  
index 5 (not included).
```

Output: ['cherry', 'orange', 'kiwi']

By leaving out the start value, the range will start at the first item:

Example:

This example returns the items from the beginning to, but NOT including, "kiwi":

```
thislist =  
["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
  
print(thislist[:4])
```

Output: ['apple', 'banana', 'cherry', 'orange']

By leaving out the end value, the range will go on to the end of the list:

Example:

This example returns the items from "cherry" to the end:

```
thislist =  
["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
  
print(thislist[2:])
```

Output: ['cherry', 'orange', 'kiwi', 'melon', 'mango']

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the list:

Example:

This example returns the items from "orange" (-4) to, but NOT including "mango" (-1):

```
thislist =  
["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[-4:-1])
```

Output: ['orange', 'kiwi', 'melon']

Check if Item Exists

To determine if a specified item is present in a list use the `in` keyword:

Example:

Check if "apple" is present in the list:

```
thislist = ["apple", "banana", "cherry"]  
if "apple" in thislist:  
    print("Yes, 'apple' is in the fruits list")
```

Output: Yes, 'apple' is in the fruits list

Change Item Value

If you need to modify the value of a particular item, you can do so by referencing its index number.

Example:

Change the second item:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1] = "blackcurrant"  
print(thislist)
```

Output: ['apple', 'blackcurrant', 'cherry']

Change a Range of Item Values

To change the value of items within a specific range, create a list with the new values and use slicing to replace the old values with the new ones.

Example:

Change the values "banana" and "cherry" with the values "blackcurrant" and "watermelon":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]  
thislist[1:3] = ["blackcurrant", "watermelon"]  
print(thislist)
```

Output: ['apple', 'blackcurrant', 'watermelon', 'orange', 'kiwi', 'mango']

If you insert more items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Example:

Change the second value by replacing it with two new values:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1:2] = ["blackcurrant", "watermelon"]  
print(thislist)
```

Output: ['apple', 'blackcurrant', 'watermelon', 'cherry']

- ❖ Note: The length of the list will change when the number of items inserted does not match the number of items replaced.

If you insert less items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Example:

Change the second and third value by replacing it with one value:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1:3] = ["watermelon"]  
print(thislist)
```

Output: ['apple', 'watermelon']

Insert Items

To add a new item to a list, without replacing any existing values, use the `insert()` method. This method inserts an item at a specific index.

Example:

Insert "watermelon" as the third item:

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(2, "watermelon")  
print(thislist)
```

Output: ['apple', 'banana', 'watermelon', 'cherry']

❖ Note: As a result of the example above, the list will now contain 4 items.

Extend List

To append elements from another list to the current list, use the `extend()` method.

Example:

Add the elements of tropical to thislist:

```
thislist = ["apple", "banana", "cherry"]  
tropical = ["mango", "pineapple", "papaya"]  
thislist.extend(tropical)  
print(thislist)
```

Output: ['apple', 'banana', 'cherry', 'mango', 'pineapple', 'papaya']

The elements will be added to the end of the list.

Add Any Iterable

The `extend()` method does not have to append lists, you can add any iterable object (tuples, sets, dictionaries etc.).

Example:

Add elements of a tuple to a list:

```
thislist = ["apple", "banana", "cherry"]  
thistuple = ("kiwi", "orange")  
thislist.extend(thistuple)  
print(thislist)
```

Output: ['apple', 'banana', 'cherry', 'kiwi', 'orange']

Remove Specified Item

The `remove()` method removes the specified item.

Example:

Remove "banana":

```
thislist = ["apple", "banana", "cherry"]  
thislist.remove("banana")  
print(thislist)
```

Output: ['apple', 'cherry']

If there are more than one item with the specified value, the `remove()` method removes the first occurrence:

Example:

Remove the first occurrence of "banana":

```
thislist = ["apple", "banana", "cherry", "banana", "kiwi"]  
thislist.remove("banana")  
print(thislist)
```

Output: ['apple', 'cherry', 'banana', 'kiwi']

Remove Specified Index

The `pop()` method removes the desired index.

Example:

Remove the second item:

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop(1)  
print(thislist)
```

Output: ['apple', 'cherry']

The `pop()` method removes the last item if you do not specify the index.

Example:

Remove the last item:

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop()  
print(thislist)
```

Output: ['apple', 'banana']

The `del` keyword also removes the specified index:

Example:

Remove the first item:

```
thislist = ["apple", "banana", "cherry"]  
del thislist[0]  
print(thislist)
```

.

Output: ['banana', 'cherry']

The `del` keyword can also delete the list completely.

Example:

Delete the entire list:

```
thislist = ["apple", "banana", "cherry"]  
del thislist  
print(thislist)
```

Output: `NameError: name 'thislist' is not defined`

Clear the List

By calling the `clear()` method, you can remove all the elements from a list. However, the list itself remains, but with no content.

Example:

Clear the list content:

```
thislist = ["apple", "banana", "cherry"]  
thislist.clear()  
print(thislist)
```

Output: []

Sort List Alphanumerically

List objects have a `sort()` method that will sort the list alphanumerically, ascending, by default:

Example:

Sort the list alphabetically:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]  
thislist.sort()  
print(thislist)
```

Output: ['banana', 'kiwi', 'mango', 'orange', 'pineapple']

Example:

Sort the list numerically:

```
thislist = [100, 50, 65, 82, 23]  
thislist.sort()  
print(thislist)
```

Output: [23, 50, 65, 82, 100]

Sort Descending

To sort descending, use the keyword argument `reverse = True`:

Example:

Sort the list descending:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]  
thislist.sort(reverse = True)  
print(thislist)
```

Output: ['pineapple', 'orange', 'mango', 'kiwi', 'banana']

Example:

Sort the list descending:

```
thislist = [100, 50, 65, 82, 23]  
thislist.sort(reverse = True)  
print(thislist)
```

Output: [100, 82, 65, 50, 23]

Case Insensitive Sort

By default, the `sort()` method is case sensitive, resulting in all capital letters being sorted before lower case letters:

Example:

Case sensitive sorting can give an unexpected result:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
thislist.sort()  
print(thislist)
```

Output: ['Kiwi', 'Orange', 'banana', 'cherry']

Luckily, we can use built-in functions as key functions when sorting a list. If we want to sort the list in a case-insensitive manner, we can pass `str.lower` as a key function.

Example:

Perform a case-insensitive sort of the list:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
thislist.sort(key = str.lower)  
print(thislist)
```

Output: ['banana', 'cherry', 'Kiwi', 'Orange']

Reverse Order

The `reverse()` method can be used to reverse the order of a list, regardless of the original sorting order of the elements.

Example:

Reverse the order of the list items:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
thislist.reverse()  
print(thislist)
```

Output: ['cherry', 'Kiwi', 'Orange', 'banana']

Copy a List

It's important to understand that you can't copy a list by simply typing `list2 = list1`. If you do this, `list2` will only be a reference to `list1`, meaning any changes made to `list1` will automatically be made to `list2` as well. If you want to create a true copy of a list, you can use the built-in List method `copy()`.

Example:

Make a copy of a list with the `copy()` method:

```
thislist = ["apple", "banana", "cherry"]  
mylist = thislist.copy()  
print(mylist)
```

Output: ['apple', 'banana', 'cherry']

Another way to make a copy is to use the built-in method `list()`.

Example:

Make a copy of a list with the `list()` method:

```
thislist = ["apple", "banana", "cherry"]  
mylist = list(thislist)  
print(mylist)
```

Output: ['apple', 'banana', 'cherry']

Join Two Lists

Concatenating lists in Python can be done using the + operator.

Example:

Join two lists:

```
list1 = ["a", "b", "c"]
```

```
list2 = [1, 2, 3]
```

```
list3 = list1 + list2
```

```
print(list3)
```

Output: ['a', 'b', 'c', 1, 2, 3]

Another way to combine two lists is to iterate through each element in list2 and append it to list1 using the append() method.

Example:

Append list2 into list1:

```
list1 = ["a", "b" , "c"]
```

```
list2 = [1, 2, 3]
```

```
for x in list2:  
    list1.append(x)
```

```
print(list1)
```

Output: ['a', 'b', 'c', 1, 2, 3]

Or you can use the `extend()` method, where the purpose is to add elements from one list to another list:

Example:

Append list2 into list1:

```
list1 = ["a", "b" , "c"]
```

```
list2 = [1, 2, 3]
```

```
list1.extend(list2)
```

```
print(list1)
```

Output: ['a', 'b', 'c', 1, 2, 3]

List Methods

There are certain built-in methods available in Python that can be used on lists.

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Python Arrays

Python Lists can be used to store multiple values in one variable instead of using built-in support for Arrays.

Example:

Create an array containing car names:

```
cars = ["Ford", "Volvo", "BMW"]
```

Output: ['Ford', 'Volvo', 'BMW']

What is an Array?

An array is a variable that can store multiple values at once. For instance, if you have a list of items (such as a list of car names), storing the cars in individual variables would be cumbersome and inefficient. Instead, you can use an array to store all the car names in one place, making it easier to manage and access them:

```
car1 = "Ford"
```

```
car2 = "Volvo"
```

```
car3 = "BMW"
```

Let's say you need to find a specific car from a list of cars, or if you have a long list of 300 cars. What do you do? Well, the solution is an array!

An array allows you to store multiple values under a single name, and you can access those values by referring to an index number.

Access the Elements of an Array

You refer to an array element by referring to the *index* number.

Example:

Get the value of the first array item:

```
cars = ["Ford", "Volvo", "BMW"]
```

```
x = cars[0]
```

Output: Ford

Example:

Modify the value of the first array item:

```
cars[0] = "Toyota"
```

Output: ['Toyota', 'Volvo', 'BMW']

The Length of an Array

Use the `len()` method to return the length of an array (the number of elements in an array).

Example:

Return the number of elements in the `cars` array:

```
cars = ["Ford", "Volvo", "BMW"]
```

```
x = len(cars)
```

Output: 3

❖ Note: The length of an array is always one more than the highest array index.

Adding Array Elements

You can use the `append()` method to add an element to an array.

Example:

Add one more element to the `cars` array:

```
cars.append("Honda")
```

Output: ['Ford', 'Volvo', 'BMW', 'Honda']

Removing Array Elements

You can use the `pop()` method to remove an element from the array.

Example:

Delete the second element of the `cars` array:

```
cars = ["Ford", "Volvo", "BMW"]  
cars.pop(1)
```

Output: ['Ford', 'BMW']

You can also use the `remove()` method to remove an element from the array.

Example:

Delete the element that has the value "Volvo":

```
cars.remove("Volvo")
```

Output: ['Ford', 'BMW']

The list's `remove()` method only removes the first occurrence of the specified value.

Array Methods

A variety of built-in methods can be used on lists/arrays in Python. These methods are readily available and can make programming more efficient and effective.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list
flatten()	Convert the array to one dimension

❖ Note: Python does not have built-in support for Arrays, but Python Lists can be used instead.

Thank you